

[AI 기반 맞춤형 정보 큐레이션 및 요약 서비스]

소프트웨어 설계서

- 주민재 -

문서번호 : [주민재]소프트웨어설계서_Doc001

소 속 : 한국항공대학교 소프트웨어학과

이 름 : 주민재

Github : https://github.com/Juminjae/26-1-SWE_AI-Curation

제/개정 이력

[illegible]

목 차

1. 서론	1
1.1 목적 및 범위	1
1.2 용어 정의	1
1.3 참조 문서	1
2. 소프트웨어 아키텍처	2
2.1 정적 구조	2
2.2 동적 구조	4
3. 모듈/패키지 설계	5
3.1 SDD-M/p-001 : 모듈 혹은 패키지 이름	5
3.2 SDD-M/P-002 : 모듈 또는 패키지 이름	5
3.3 SDD-M/P-00x : 모든 모듈/패키지에 대하여 작성	5
4. 인터페이스 설계	8
4.1 외부 시스템 인터페이스	8
4.2 사용자 인터페이스	8
5. 데이터 설계	9
6. 구현 기술 설계	10
7. 요구사항 추적표	11
8. 부록	12

1. 서론

1.1 목적 및 범위

본 소프트웨어 설계서는 'AI 기반 맞춤형 정보 큐레이션 및 요약 서비스' 개발 프로젝트의 구체적인 설계 산출물을 정의하기 위해 작성되었습니다. 요구사항 정의서 및 요구사항 분석서에서 도출된 기능적·비기능적 요구사항을 실제 구현 가능한 설계로 구체화하며, 시스템 아키텍처, 모듈, 데이터베이스, API, 인터페이스, 보안 및 배포 설계를 포함합니다.

본 문서는 다음 범위에 적용됩니다.

- 시스템 아키텍처 및 계층별 구성 요서 설계
- 백엔드(Spring Boot), 프론트엔드(React Native), AI 엔진, 인프라(AWS) 모듈 설계
- PostgreSQL 기반 데이터베이스 스키마 및 ERD 설계
- RESTful API 명세 및 외부 인터페이스 설계
- 보안 및 AWS 기반 배포 설계
- 요구사항 추적표

1.2 대상 시스템 개요

사용자가 등록한 관심 키워드를 바탕으로 뉴스 및 기술 칼럼 데이터를 자동 크롤링하고, LLM API(GPT-4o)를 활용해 핵심 내용을 3줄로 요약하여 개인형 모바일 대시보드로 시각화 제공하는 지능형 지식 관리 시스템입니다. React Native으로 동작하며 Spring Boot와 AWS 클라우드 인프라 위에서 운영되어집니다.

1.3 용어 정의

용어	설명
LLM	OpenAI의 GPT-4o를 활용한 텍스트 요약 및 키워드 추출 엔진
Crawling	RSS 및 웹 페이지에서 정기적으로 데이터를 수집하는 프로세스
Spring Boot	백엔드 비즈니스 로직 및 API 서버 프레임워크
React Native	모바일 사용자 인터페이스 구축을 위한 프레임워크

1.4 참조 문서

[주민재] 시스템정의서_260316_Doc-001.pdf

[주민재] 프로젝트관리계획서_260413_Doc-001.pdf

[주민재] 요구사항정의서_260518_Doc-001.pdf

[주민재] 요구사항분석서_260518_Doc-001.pdf

2. 소프트웨어 아키텍처

2.1 정적 구조(Static Architecture)

정적 구조는 시스템을 구성하는 컴포넌트·패키지·클래스의 배치와 이들 사이의 의존 관계를 나타냅니다. 실행 시점에 무관하게 소스코드 수준에서 확인할 수 있는 구조를 기술합니다.

2.1.1 컴포넌트 구성 및 의존 관계

시스템은 10개의 주요 컴포넌트로 구성되며, C-01(MobileApp)이 C-02(APIServer)를 호출하는 클라이언트-서버 구조를 핵심으로 합니다. C-02는 내부적으로 C-03(크롤링), C-04(AI 요약), C-05(DB), C-06(추천 엔진)을 조율하며, C-04-C-06은 외부 서비스인 C-08(OpenAI)·C-09(LangChain)와 연동합니다.

ID	컴포넌트명	역할 / 책임	인터페이스 제공	의존 컴포넌트
C-01	MobileApp (React Native)	사용자 UI 렌더링, 키워드 설정, 대시보드·스크랩·알림 화면 제공	사용자 인터랙션 이벤트	C-02 (API Server)
C-02	APIServer (Spring Boot)	비즈니스 로직 처리, REST API 제공, 크롤링·AI 요약·추천 엔진 조율	RESTful API (JSON/HTTPS)	C-03, C-04, C-05, C-06
C-03	CrawlingEngine	배치 스케줄러로 외부 뉴스·RSS 자동 수집, 메타데이터 추출	크롤링 결과 DTO	C-05 (DB), C-07 (외부 RSS)
C-04	LLMService (AI 요약 엔진)	원문 프롬프트 엔지니어링, GPT-4o API 호출, 3줄 요약 생성 및 키워드 추출	Summary DTO	C-08 (OpenAI API), C-05
C-05	CloudDB (PostgreSQL/RDS)	사용자·키워드·기사·요약·스크랩·피드백·선호도·알림 데이터 영속 관리	JDBC / JPA Repository	-
C-06	Recommendation	LangChain 기반 사용자 피	필터링된 Summary	C-05, C-09

	Engine	드백 가중치 학습, 대시보드 노출 순위 필터링	리스트	(LangChain)
C-07	외부 뉴스·RSS 서버	크롤링 대상 외부 뉴스·기술 칼럼 RSS 피드 및 웹 페이지 제공	HTTP/RSS XML	-
C-08	OpenAI GPT-4o API	원문 텍스트 수신 후 3줄 요약 및 핵심 키워드 반환	REST API (JSON/HTTPS)	-
C-09	LangChain 모듈	피드백 로그 기반 사용자 선호도 모델 학습 및 가중치 업데이트	Python 내부 API	C-05
C-10	FCM 푸시 서버	Firebase Cloud Messaging으로 사용자 모바일 디바이스에 푸시 알림 발송	REST API (HTTPS)	-

2.1.2 패키지 / 레이어 구조

백엔드는 Controller → Service → Repository → Domain의 계층형 패키지 구조를 따르며, 각 계층은 단방향 의존만 허용합니다. 프론트엔드는 screens / components / services 로 분리하여 화면 로직과 API 호출 레이어를 명확히 구분합니다.

패키지 / 레이어	주요 클래스·모듈	설명
com.aicuration.api.controller	KeywordController, DashboardController, ScrapController, FeedbackController, NotificationController, AuthController	HTTP 요청 수신, 응답 반환 (Presentation Layer)
com.aicuration.api.service	CrawlingService, LLMService, RecommendationService, ScrapService, FeedbackService,	핵심 비즈니스 로직 처리 (Business Layer)

	NotificationService	
com.aicuration.api.repository	UserRepository, KeywordRepository, ArticleRepository, SummaryRepository, ScrapRepository, FeedbackRepository	JPA 기반 DB 접근 (Data Access Layer)
com.aicuration.api.domain	User, Keyword, Article, Summary, Scrap, Feedback, UserPreference, Notification	JPA 엔티티 (도메인 모델)
com.aicuration.api.security	JwtTokenProvider, JwtAuthFilter, SecurityConfig	JWT 인증·인가 필터 체인
com.aicuration.api.batch	CrawlingBatchJob, SummaryBatchJob, NotificationBatchJob	Spring Batch 배치 잡 스케줄러
com.aicuration.api.dto	KeywordRequest/Response, DashboardResponse, SummaryDto, FeedbackRequest	계층 간 데이터 전달 객체 (DTO)
screens/ (React Native)	HomeScreen, KeywordScreen, DashboardScreen, ScrapScreen, SettingsScreen	모바일 화면 단위 컴포넌트
components/ (React Native)	SummaryCard, TrendChart, KeywordBadge, FeedbackButton, ScrapIcon	재사용 UI 컴포넌트
services/ (React Native)	apiService.js, authService.js, notificationService.js	Axios 기반 API 호출 레이어

2.1.3 클래스 다이어그램 - 주요 엔티티 관계

도메인 모델 내 주요 클래스(엔티티) 간의 관계는 다음과 같으며, 각 관계는 JPA 어노테이션

(@OneToMany, @ManyToOne 등)으로 구현됩니다.

클래스(엔티티)	관계 유형	대상 클래스	설명
User	1 : N	Keyword	사용자는 여러 관심 키워드를 등록할 수 있다.
User	1 : 1	UserPreference	사용자는 하나의 선호도 모델 파라미터를 보유한다.
User	1 : 1	Notification	사용자는 하나의 푸시 알림 설정을 가진다.
Keyword	1 : N	Article	키워드 기반으로 여러 기사가 수집된다.
Article	1 : 1	Summary	수집된 기사 1건은 하나의 3줄 요약본을 가진다.
User	N : M	Summary (via Scrap)	사용자는 여러 요약을 스크랩하며, Scrap 테이블이 중간 매핑 역할을 한다.
User	N : M	Summary (via Feedback)	사용자는 각 요약 카드에 좋아요/싫어요 피드백을 남기며, Feedback 테이블이 매핑한다.
LLMService	의존(uses)	CrawlingEngine	크롤링 완료 후 LLMService가 미요약 Article 목록을 호출하여 요약을 진행한다.
RecommendationEngine	의존(uses)	Feedback, UserPreference	피드백 데이터를 읽어 UserPreference의 가중치를 업데이트한다.

2.2 동적 구조(Dynamic Architecture)

정적 구조는 시스템을 구성하는 컴포넌트·패키지·클래스의 배치와 이들 사이의 의존 관계를 나타냅니다. 실행 시점에 무관하게 소스코드 수준에서 확인할 수 있는 구조를 기술합니다.

2.2.1 시퀀스 다이어그램 SF-01: 크롤링 -> 요약 -> 저장 -> 알림 -> 대시보드(메인플로우)

시스템의 핵심 데이터 파이프라인으로, 배치 스케줄러 기동부터 사용자가 대시보드를 조회하기까지의 전체 플로우를 나타냅니다.

순서	송신자	수신자	메시지 / 처리 내용
----	-----	-----	-------------

1	Scheduler (Batch)	CrawlingEngine	매일 지정 시간에 startCrawling(activeKeywords) 호출
2	CrawlingEngine	CloudDB (PostgreSQL)	SELECT activated keywords → 활성화 키워드 목록 반환
3	CrawlingEngine	외부 뉴스·RSS 서버	[for each keyword] parseRSS() / parseHTML() → 기사 메타데이터(제목, 본문, URL, 발행일) 추출
4	CrawlingEngine	CloudDB	INSERT INTO articles (중복 URL skip)
5	Scheduler	LLMService	startSummarization() 호출 (크롤링 완료 후 트리거)
6	LLMService	CloudDB	SELECT articles WHERE summary IS NULL → 미요약 기사 목록 조회
7	LLMService	OpenAI GPT-4o API	[for each article] buildPrompt() → callGPT4o(prompt) → 3줄 요약 + 핵심 키워드 반환
8	LLMService	CloudDB	INSERT INTO summaries (summary_text, extracted_keywords, article_id)
9	NotificationService	CloudDB	SELECT users WHERE scheduled_time = NOW() → 알림 대상 사용자 조회
10	NotificationService	FCM 서버	send(fcmToken, summarySnapshot) → 푸시 알림 발송
11	MobileApp	APIServer	GET /dashboard (Authorization: Bearer JWT) → 대시보드 데이터 요청
12	APIServer	RecommendationEngine	rankSummaries(userId) → 선호도 가중치 기반 정렬된 요약 리스트 반환
13	RecommendationEngine	CloudDB	SELECT summaries + user_preferences WHERE userId → 데이터 조회 및 필터링 연산
14	APIServer	MobileApp	200 OK { summaries[], trendKeywords[] } 3초 이내 응답

2.2.2 시퀀스 다이어그램 SF-02: 피드백 처리 -> 선호도 갱신

사용자가 요약 카드에 좋아요/싫어요 피드백을 남기면, 백그라운드에서 LangChain 학습이 비동기적으로 진행되어 user_preferences가 업데이트됩니다.

순서	송신자	수신자	메시지 / 처리 내용
1	사용자	MobileApp	요약 카드 하단 좋아요 / 싫어요 버튼 탭
2	MobileApp	APIServer	POST /feedback { summaryId, type: LIKE DISLIKE } (Authorization: Bearer JWT)
3	APIServer	CloudDB	INSERT INTO feedbacks (userId, summaryId, type, created_at)
4	APIServer	RecommendationEngine	learnPreference(feedback) 비동기 호출 (즉시 응답 후 백그라운드 처리)
5	RecommendationEngine	LangChain 모듈	trainWeights(feedbackLogs) → 사용자 선호도 가중치 모델 학습
6	RecommendationEngine	CloudDB	UPDATE user_preferences SET topic_weights = {...} WHERE userId
7	APIServer	MobileApp	200 OK { feedbackId, updatedAt } - UI 피드백 아이콘 즉시 업데이트
8 (예외)	MobileApp	LocalQueue	[네트워크 오류 시] 피드백 요청 로컬 큐에 임시 저장 → 복구 시 재전송 (UX 보존)

2.2.3 시퀀스 다이어그램 SF-03: 스크랩 저장

사용자가 요약 카드를 보관함에 저장하는 단방향 흐름으로, 중복 스크랩 방지 로직을 포함합니다.

순서	송신자	수신자	메시지 / 처리 내용
1	사용자	MobileApp	대시보드의 요약 카드에서 스크랩(북마크) 아이콘 탭
2	MobileApp	APIServer	POST /scraps { summaryId } (Authorization: Bearer JWT)
3	APIServer	CloudDB	validateJWT() → userId 추출 후 중복 스크랩 확인 (SELECT 1 FROM scraps WHERE userId AND summaryId)
4a (정상)	CloudDB	APIServer	INSERT INTO scraps (userId, summaryId, savedAt) RETURNING scrapId - 저장 성공
4b (중복)	CloudDB	APIServer	이미 존재하는 스크랩 → 409 Conflict 반환, MobileApp에 "이미 스크랩된 정보입니다" 토스트 표시
5	APIServer	MobileApp	201 Created { scrapId, savedAt } → 스크랩 아이콘 시각적 강조(filled)

2.2.4 시퀀스 다이어그램 SF-04: 키워드 등록

사용자가 모바일 앱에서 관심 키워드를 등록하는 흐름으로, 공란 검증(로컬)과 중복 검증(서버)을 분리합니다.

순서	송신자	수신자	메시지 / 처리 내용
1	사용자	MobileApp	키워드 입력 후 '등록' 버튼 탭
2	MobileApp	MobileApp (로컬 검증)	[입력값 공란 시] 등록 실패 메시지 표시 - API 호출 없이 즉시 처리

3	MobileApp	APIServer	POST /keywords { word } (Authorization: Bearer JWT)
4	APIServer	CloudDB	중복 키워드 확인 (SELECT 1 FROM keywords WHERE userId AND word)
5a (정상)	CloudDB	APIServer	INSERT INTO keywords (userId, word, isActive=true) – 저장 성공
5b (중복)	CloudDB	APIServer	409 Conflict → "이미 등록된 키워드입니다" 오류 응답
6	APIServer	MobileApp	201 Created { keywordId, word } → 성공 메시지 표시 후 대시보드 화면으로 이동

2.2.5 상태 다이어그램

시스템 내 주요 객체(Article, Scrap, Feedback, Notification)의 생명주기와 상태 전환 조건을 정의합니다.

대상 객체	상태	전환 트리거 (→ 다음 상태)	설명
Article	CRAWLED	크롤링 엔진이 기사 수집 완료 → PENDING_SUMMARY	수집 완료, 아직 요약 미생성 상태
	PENDING_SUMMARY	LLMService 요약 성공 → SUMMARIZED / API 오류 → SUMMARY_FAILED	GPT-4o 요약 대기 중
	SUMMARIZED	대시보드 조회 → DISPLAYED	3줄 요약 및 키워드 저장 완료
	SUMMARY_FAILED	재시도 스케줄러 → PENDING_SUMMARY	타임아웃·API 오류로 요약 보류
Scrap (보관함)	SAVED	사용자 스크랩 탭 → SAVED / 삭제 탭 → DELETED	보관함에 저장된 상태, 영구 유지
	DELETED	(최종 상태)	사용자 삭제 요청으로 레코드 제거
Feedback	SUBMITTED	피드백 등록 → SUBMITTED → 학습 반영 → APPLIED	피드백 DB 저장 완료

	APPLIED	(최종 상태)	RecommendationEngine 학습에 반영 완료
Notification (푸시 알림)	SCHEDULED	지정 시간 도달 → TRIGGERED	발송 예약 상태 (사용자 설정 시간 대기)
	TRIGGERED	FCM 발송 성공 → SENT / 권한 차단 → BLOCKED	스케줄러가 FCM 발송 시도
	SENT	(최종 상태)	FCM 정상 발송 완료
	BLOCKED	(최종 상태)	OS 알림 권한 차단으로 디바이스에 미노출

3. 모듈/패키지 설계

3.1 SDD-M-001 : Crawling

3.1.1 모듈 설명

먼저 소프트웨어를 구성하는 패키지에 대하여 간단히 정의합니다. 예를 들면 다음과 같습니다.

패키지설명	사용자가 등록한 관심 키워드를 기반으로 외부 뉴스·기술 칼럼·RSS 데이터를 자동으로 수집하고 메타데이터(제목, 본문, URL, 발행일)를 추출하는 패키지	
구성 클래스	CrawlingService	배치 스케줄러를 구동하고 크롤링 전체 흐름 조율 클래스
	RssParser	RSS XML 피드를 파싱하여 기사 목록 추출 클래스
	HtmlParser	웹 페이지 HTML에서 본문 추출 클래스
	ArticleRepository	수집된 기사 엔티티를 DB 저장·조회 JPA Repository

3.1.2 구성 클래스 설계

(1) CM-001 : CrawlingService

클래스 타입	Concrete	관련 Use Case	U_02
설 명	배치 스케줄러(@Scheduled)로 매일 지정 시간에 활성화 키워드 목록을 DB에서 조회하고, RSS/HTML 파서를 호출하여 외부 뉴스 데이터를 수집한다.		
멤버 변수	- schedule Time: LocalTime - TargetDomains: List<String>		
멤버 함수	+ startCrawling(keywords: List<String>): void + fetchArticle(keyword: String): List<Article>		
관계성	Generalization(a-kind-of): Aggregation (has-parts) : RssParser, HtmlParser Other Associations :ArticleRepository, KeywordRepository		

(2) CM-002: RssParser

클래스 타입	Concrete	관련 Use Case	U_02
설 명	대상 URL의 RSS XML 피드를 파싱하여 기사 제목, 본문 요약, 원본 URL, 발행일을 추출한다.		
멤버 변수	- timeoutMs : Integer		
멤버 함수	+ parseRss(url: String) : List<Article>		
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : CrawlingService		

3.1.3 패키지 행위

배치 스케줄러 → CrawlingService.startCrawling() → [활성 키워드 조회] → [for each domain]
RssParser.parseRSS() /HtmlParser.parse() → 중복 URL 체크 → ArticleRepository.save()

3.1.4 멤버함수(메소드) 설계

(1) M-001 : startCrawling()

소속 클래스	CM-001 : CrawlingService
트리거 클래스	Scheduler (@Scheduled)
파라미터	activeKeywords : List<String>
세부 처리 로직	① 배치 스케줄러가 매일 지정 시간에 메서드를 호출한다. ② KeywordRepository에서 isActive=true인 키워드 목록을 조회한다. ③ 키워드별로 사전 정의된 뉴스 도메인을 순회하며 RSS 파싱 또는 HTML 크롤링을 수행한다. ④ 추출된 기사의 source_url이 DB에 존재하면 skip, 신규이면 articles 테이블에 INSERT한다. ⑤ 접근 불가 도메인은 오류 로그 기록 후 다음 도메인으로 계속 진행한다

(2)M-002 : fetchArticles()

소속 클래스	CM-001 : CrawlingService
--------	--------------------------

트리거 클래스	CM-001 : CrawlingService (내부 호출)
파라미터	keyword : String
세부 처리 로직	① RssParser.parseRSS() 또는 HtmlParser.parse()를 호출하여 기사 목록을 수집한다. ② 각 기사의 제목, 본문, URL, 발행일, 이미지 URL을 Article 객체로 매핑하여 반환한다.

3.2 SDD-M/P-002 : LLMSummary

3.2.1 패키지 설명

패키지 설명	크롤링으로 수집된 원문 텍스트를 프롬프트 엔지니어링 후 OpenAI GPT-4o API에 전달하여 3줄 요약본과 핵심 키워드를 생성·저장하는 패키지
구성 클래스	LLMService: 프롬프트 구성, GPT-4o API 호출, 요약 결과 저장을 담당하는 클래스 PromptBuilder: 원문 텍스트를 3줄 요약 요청 프롬프트로 변환하는 유틸리티 클래스 SummaryRepository: 생성된 요약본을 DB에 저장·조회하는 JPA Repository

3.2.2 구성 클래스 설계

(1)CM-003 : LLMService

클래스 타입	Concrete
관련 Use Case	U_03
설 명	미요약 기사를 DB에서 조회한 후 PromptBuilder로 프롬프트를 구성하고, OpenAI GPT-4o API를 호출하여 3줄 요약본과 핵심 키워드를 받아 summaries 테이블에 저장한다.
멤버 변수	- openAiApiKey : String (환경 변수) - promptTemplate : String - timeoutSec : Integer
멤버 함수	+ startSummarization() : void + summarize(article: Article) : Summary + callGPT4o(prompt: String) : String + extractKeywords(text: String) : List<String>
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : PromptBuilder

	Other Associations : ArticleRepository, SummaryRepository, OpenAI API
--	---

3.2.3 패키지 행위

Scheduler트리거 → LLMService.startSummarization() → [미요약 기사 조회] → [foreach article]
 PromptBuilder.build() → callGPT4o(prompt) → 3줄 요약 + 키워드 파싱 →
 SummaryRepository.save()

3.2.4 멤버함수(메소드) 설계

(1)M-003 : summarize()

소속 클래스	CM-003 : LLMService
트리거 클래스	Scheduler (크롤링 완료 후 트리거)
파라미터	article : Article
세부 처리 로직	① PromptBuilder로 원문 본문을 3줄 요약 요청 프롬프트로 변환한다. ② OpenAI GPT-4o API를 호출하고 응답을 수신한다. ③ 타임아웃(30초) 발생 시 Exponential Backoff로 최대 3회 재시도한다. ④ 최종 실패 시 해당 기사는 SUMMARY_FAILED 상태로 기록한다. ⑤ 성공 시 3줄 요약본과 핵심 키워드를 summaries 테이블에 저장한다.

3.3 SDD-M/P-003 : Dashboard

3.3.1 패키지 설명

패키지 설명	당일 수집·요약된 최신 정보를 가독성이 높은 카드·트렌드 차트 형태로 React Native 앱 화면에 3초 이내 렌더링하는 패키지
구성 클래스	DashboardController 대시보드 데이터 요청을 수신하고 응답을 반환하는 REST 컨트롤러 DashboardService 추천 엔진과 연동하여 정렬된 요약 카드 리스트와 트렌드 키워드를 조합하는 서비스

3.3.2 구성 클래스 설계

(1)CM-004 : DashboardService

클래스 타입	Concrete
관련 Use Case	U_04

설 명	RecommendationService로부터 개인화 정렬된 요약 카드 목록을 받고, DB에서 트렌드 키워드 빈도수를 집계하여 대시보드 응답 DTO를 구성한다.
멤버 변수	- responseTimeoutMs : Integer
멤버 함수	+ getDashboard(userId: Long) : DashboardResponse + getTrendKeywords() : List<String>
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : RecommendationService, SummaryRepository

3.3.3 패키지 행위

MobileApp → GET /dashboard → DashboardController → DashboardService.getDashboard() → RecommendationService.rankSummaries() → CloudDB 조회 → DashboardResponse 반환 (3초 이내)

3.3.4 멤버함수(메소드) 설계

(1)M-004 : getDashboard()

소속 클래스	CM-004 : DashboardService
트리거 클래스	DashboardController (GET /dashboard 요청)
파라미터	userId : Long
세부 처리 로직	① JWT에서 userId를 추출하여 인증을 검증한다. ② RecommendationService.rankSummaries(userId)를 호출하여 선호도 기반 정렬된 요약 카드 목록을 받는다. ③ SummaryRepository에서 당일 트렌드 키워드 빈도수 Top-10을 집계한다. ④ summaries와 trendKeywords를 DashboardResponse DTO로 조합하여 반환한다. ⑤ 서버 응답은 요청 접수 후 최대 3초 이내에 완료되어야 한다 (NFR-003).

3.4 SDD-M/P-004 : Scrap

3.4.1 패키지 설명

패키지 설명	사용자가 중요하다고 판단한 요약 카드를 보관함에 영구 저장하고, 원본 URL 이동 및 삭제 기능을 제공하는 패키지
--------	---

구성 클래스	ScrapService	스크랩 저장·조회·삭제 비즈니스 로직 처리 클래스
	ScrapRepository	Scrap 엔티티를 DB에 저장·조회·삭제하는 JPA Repository
	Repository	

3.4.2 구성 클래스 설계

(1)CM-005 : ScrapService

클래스 타입	Concrete
관련 Use Case	U_05
설 명	사용자의 스크랩 저장 요청을 받아 중복 여부를 확인하고, 신규이면 scraps 테이블에 삽입한다. 삭제 요청 시 해당 레코드를 제거하며, 보관함 목록을 조회하여 반환한다.
멤버 변수	
멤버 함수	+ save(userId: Long, summaryId: Long) : Scrap + delete(userId: Long, scrapId: Long) : void + getList(userId: Long) : List<Scrap>
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : ScrapRepository, SummaryRepository

3.4.3 패키지 행위

MobileApp→ POST /scraps {summaryId} → ScrapController → ScrapService.save() → [중복 확인]
→ scraps INSERT → 201 Created / 409Conflict 반환

3.4.4 멤버함수(메소드) 설계

(1)M-005 : save()

소속 클래스	CM-005 : ScrapService
트리거 클래스	ScrapController (POST /scraps 요청)
파라미터	userId : Long, summaryId : Long
세부 처리 로직	① (userId, summaryId)로 scraps 테이블에 중복 레코드가 있는지 확인한다. ② 이미 존재하면 409 Conflict를 반환하고 '이미 스크랩된 정보입니다' 토스트를 표시한다.

	③ 신규이면 scraps 테이블에 INSERT하고 201 Created와 scrapId, savedAt을 반환한다.
--	--

3.5 SDD-M/P-005 : Feedback

3.5.1 패키지 설명

패키지 설명	사용자가 요약 카드에 남긴 좋아요/싫어요 피드백을 수집하고 DB에 기록하는 패키지
구성 클래스	FeedbackService: 피드백 등록 비즈니스 로직 처리 클래스 FeedbackRepository: Feedback 엔티티를 DB에 저장·조회하는 JPA Repository

3.5.2 구성 클래스 설계

(1)CM-006 : FeedbackService

클래스 타입	Concrete
관련 Use Case	U_07
설 명	사용자 피드백(LIKE/DISLIKE)을 수신하여 feedbacks 테이블에 저장하고, 비동기적으로 RecommendationService에 선호도 학습을 위임한다.
멤버 변수	
멤버 함수	+ submit(userId: Long, summaryId: Long, type: FeedbackType) : Feedback
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : FeedbackRepository, RecommendationService

3.5.3 패키지 행위

MobileApp → POST /feedback {summaryId, type} → FeedbackController → FeedbackService.submit() → feedbacks INSERT → [비동기] RecommendationService.learnPreference() → 200 OK

3.5.4 멤버함수(메소드) 설계

(1)M-006 : submit()

소속 클래스	CM-006 : FeedbackService
--------	--------------------------

트리거 클래스	FeedbackController (POST /feedback 요청)
파라미터	userId: Long, summaryId: Long, type: FeedbackType(LIKE DISLIKE)
세부 처리 로직	① feedbacks 테이블에 피드백 레코드를 INSERT한다. ② @Async로 RecommendationService.learnPreference(feedback)을 비동기 호출하여 선호도 모델을 업데이트한다. ③ 즉시 200 OK { feedbackId, updatedAt }를 반환하여 UX를 보존한다. ④ 네트워크 오류 시 앱은 피드백 요청을 로컬 큐에 임시 저장하고 연결 복구 시 재전송한다.

3.6 SDD-M/P-006 : Recommendation

3.6.1 패키지 설명

패키지 설명	LangChain 기반으로 사용자 피드백 가중치를 학습하고, 대시보드 요약 카드 노출 순위를 개인 선호도에 맞게 필터링하는 패키지
구성 클래스	RecommendationService: 선호도 학습 및 대시보드 정렬 비즈니스 로직 처리 클래스 LangChainModule: Python LangChain 라이브러리를 호출하여 가중치를 학습하는 어댑터 클래스

3.6.2 구성 클래스 설계

(1)CM-007 : RecommendationService

클래스 타입	Concrete
관련 Use Case	U_08
설 명	피드백 로그를 수집하여 LangChainModule로 선호도 모델을 학습하고 user_preferences를 갱신한다. 대시보드 조회 시 학습된 가중치를 기반으로 요약 카드를 정렬·필터링하여 반환한다.
멤버 변수	- weightMatrix : Map<String, Float>
멤버 함수	+ learnPreference(feedback: Feedback) : void + rankSummaries(userId: Long) : List<Summary>
관계성	Generalization(a-kind-of) :

	Aggregation (has-parts) : LangChainModule Other Associations : FeedbackRepository, UserPreferenceRepository
--	--

3.6.3 패키지 행위

[비동기] FeedbackService → learnPreference() → LangChainModule.trainWeights() → user_preferences UPDATE [대시보드 조회 시] DashboardService → rankSummaries(userId) → user_preferences 조회 → 가중치 기반 정렬 → 상위 순서 반환

3.6.4 멤버함수(메소드) 설계

(1)M-007 : rankSummaries()

소속 클래스	CM-007 : RecommendationService
트리거 클래스	DashboardService (대시보드 조회 시)
파라미터	userId : Long
세부 처리 로직	① user_preferences에서 해당 userId의 topic_weights(JSONB)를 조회한다. ② Cold Start(피드백 0건)이면 최신 발행일 기준으로 기본 정렬한다. ③ 가중치가 있으면 각 Summary의 extracted_keywords와 가중치 매트릭스를 내적(dot product)하여 스코어를 계산한다. ④ 스코어 내림차순으로 정렬된 Summary 리스트를 반환한다.

3.7 SDD-M/P-007 : Notification

3.7.1 패키지 설명

패키지 설명	사용자가 설정한 커스텀 시간에 당일 요약된 뉴스레터 스냅샷을 FCM 서버를 통해 모바일 디바이스로 푸시 발송하는 패키지
구성 클래스	NotificationService: 푸시 알림 스케줄러 구동 및 FCM 발송 처리 클래스 NotificationRepository: Notification 엔티티(알림 설정) DB 저장·조회 클래스

3.7.2 구성 클래스 설계

(1)CM-008 : NotificationService

클래스 타입	Concrete
관련 Use Case	U_06
설 명	매 분마다 notifications 테이블에서 scheduled_time이 현재 시각인 사용자를

	조회하고, 당일 요약 스냅샷을 구성하여 FCM 서버를 통해 푸시 알림을 발송한다.
멤버 변수	- fcmServerKey : String (환경 변수)
멤버 함수	+ scheduleNotification() : void + send(fcmToken: String, message: String) : void
관계성	Generalization(a-kind-of) : Aggregation (has-parts) : Other Associations : NotificationRepository, SummaryRepository, FCM 서버

3.7.3 패키지 행위

Scheduler(매 분) → NotificationService.scheduleNotification() → [알림 대상 사용자 조회] → 요약 스냅샷 구성 → FCM.send(fcmToken, message) → SENT 상태 기록

3.7.4 멤버함수(메소드) 설계

(1)M-008 : scheduleNotification()

소속 클래스	CM-008 : NotificationService
트리거 클래스	Scheduler (@Scheduled, 매 분 실행)
파라미터	없음
세부 처리 로직	① 매 분 notifications 테이블에서 scheduled_time = 현재 시각이고 is_enabled=true 인 사용자를 조회한다. ② 각 사용자의 당일 상위 요약 스냅샷(제목 + 3줄 요약)을 구성한다. ③ Firebase Admin SDK를 통해 사용자의 FCM 토큰으로 푸시 알림을 발송한다. ④ OS 알림 권한이 차단된 경우 서버 단 트리거는 실행되나 디바이스에서 미표시된다.

4. 인터페이스 설계

4.1 외부 시스템 인터페이스

메시지 명	송신 모듈	수신 모듈	메시지 형식	전송방식
키워드 동기화	MobileApp	APIServer	JSON (REST)	HTTPS POST
크롤링 결과	CrawlingEngine	CloudDB	JPA Entity	JDBC/JPA
요약 요청	LLMService	GPT-4o	JSON (REST)	HTTPS POST
요약 저장	LLMService	CloudDB	JPA Entity	JDBC/JPA
대시보드 조회	MobileApp	APIServer	JSON (REST)	HTTPS GET
피드백 등록	MobileApp	APIServer	JSON (REST)	HTTPS POST
선호도 학습	RecommendationEngine	LangChain	Python 내부 호출	Library
푸시 알림 발송	NotificationService	FCM 서버	JSON (REST)	HTTPS POST

4.2 사용자 인터페이스

모바일 앱(React Native)의 주요 화면은 다음과 같이 구성된다

화면 명	기능 설명	관련 Use Case
키워드 설정 화면	관심 키워드 등록·수정·삭제 및 DB 동기화	U_01
대시보드 화면	요약 카드 목록, 트레드 키워드 차트, 좋아요/싫어요 피드백 버튼 표시	U_04, U_07
보관함 화면	스크랩된 요약 카드 목록 조회, 원본 URL 이동, 삭제	U_05
알림 설정 화면	뉴스레터 수신 시간 커스텀 설정 및 알림 On/Off	U_06
로그인/회원가입 화면	이메일·비밀번호 입력, JWT 토큰 발급	-

5. 데이터 설계

(1)DD-01, users — 사용자 정보

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	user_id	BIGSERIAL	PK, NOT NULL	auto	사용자 고유 ID (자동 증가)
2	email	VARCHAR(25)	UNIQUE, NOT	—	로그인 이메일

		5)	NULL		
3	password	VARCHAR(255)	NOT NULL	—	BCrypt 해시 암호화
4	fcm_token	VARCHAR(512)	NULL	—	FCM 디바이스 토큰
5	created_at	TIMESTAMP	NOT NULL	NOW()	계정 생성 일시

(2)DD-02, keywords — 관심 키워드

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	keyword_id	BIGSERIAL	PK, NOT NULL	auto	키워드 고유 ID
2	user_id	BIGINT	FK → users, NOT NULL	—	소유 사용자
3	word	VARCHAR(100)	NOT NULL	—	관심 키워드 텍스트
4	is_active	BOOLEAN	NOT NULL	TRUE	키워드 활성화 여부

(3)DD-03, articles — 크롤링 기사 원문

번호	필드 명	자료타입	값의 범주	초기 값	비고
1	article_id	BIGSERIAL	PK, NOT NULL	auto	기사 고유 ID
2	title	VARCHAR(500)	NOT NULL	—	기사 제목
3	original_content	TEXT	NULL	—	원문 본문 전체
4	source_url	VARCHAR(2048)	UNIQUE, NOT NULL	—	원본 URL (중복 방지 키)

5	published_date	DATE	NULL	—	기사 발행일
6	image_url	VARCHAR(2048)	NULL	—	대표 이미지 URL
7	keyword_id	BIGINT	FK keywords, NOT NULL	→ —	매핑 키워드
8	summary_status	VARCHAR(20)	NOT NULL	PENDING	요약 상태 (PENDING/SUMMARIZED/FAILED)

(4)DD-04, summaries — AI 3줄 요약본

번호 <o:p></o:p>	필드 명<o:p></o:p>	자료타입 <o:p></o:p>	값의 범주 <o:p></o:p>	초기 값 <o:p></o:p>	비고 <o:p></o:p>
1<o:p></o:p>	summary_id<o:p></o:p>	BIGSERIAL<o:p></o:p>	PK, NOT NULL<o:p></o:p>	auto<o:p></o:p>	요약 고유 ID<o:p></o:p>
2<o:p></o:p>	summary_text<o:p></o:p>	TEXT<o:p></o:p>	NOT NULL<o:p></o:p>	—<o:p></o:p>	GPT-4o 생성 3줄 요약<o:p></o:p>

3<o:p></o:p>	extracted_keywords<o:p></o:p>	TEXT[]<o:p></o:p>	NULL<o:p></o:p>	-<o:p></o:p>	AI 추출 핵심 키워드 배열<o:p></o:p>
4<o:p></o:p>	article_id<o:p></o:p>	BIGINT<o:p></o:p>	FK → articles, NOT NULL<o:p></o:p>	-<o:p></o:p>	원문 기사 참조 (1:1)<o:p></o:p>
5<o:p></o:p>	created_at<o:p></o:p>	TIMESTAMP<o:p></o:p>	NOT NULL<o:p></o:p>	NOW()<o:p></o:p>	요약 생성 일시<o:p></o:p>

<o:p> </o:p>

(5)DD-05, scraps – 보관함<o:p></o:p>

번호<o:p></o:p>	필드 명<o:p></o:p>	자료타입<o:p></o:p>	값의 범주<o:p></o:p>	초기 값<o:p></o:p>	비고<o:p></o:p>
1<o:p></o:p>	scrap_id<o:p></o:p>	BIGSERIAL<o:p></o:p>	PK, NOT NULL<o:p></o:p>	auto<o:p></o:p>	스크랩 고유 ID<o:p></o:p>

>					
2<o:p></o:p>>	user_id<o:p></o:p>	BIGINT<o:p></o:p>	FK → users, NOT NULL<o:p></o:p>>	—<o:p></o:p>	스크랩한 사용자<o:p></o:p>
3<o:p></o:p>>	summary_id<o:p></o:p>	BIGINT<o:p></o:p>	FK → summaries, NOT NULL<o:p></o:p>>	—<o:p></o:p>	스크랩된 요약 카드<o:p></o:p>
4<o:p></o:p>>	saved_at<o:p></o:p>	TIMESTAMP<o:p></o:p>	NOT NULL<o:p></o:p>>	NOW()<o:p></o:p>	저장 일시<o:p></o:p>

<o:p> </o:p>

(6)DD-06, feedbacks – 피드백<o:p></o:p>

번호 <o:p></o:p>>	필드 명<o:p></o:p>	자료타입 <o:p></o:p>	값의 범주 <o:p></o:p>	초기 값 <o:p></o:p>	비고 <o:p></o:p>
1<o:p></o:p>>	feedback_id<o:p></o:p>	BIGSERIAL<o:p></o:p>	PK, NOT	auto<o:p></o:p>	피드백 고유

p>< </o:p> >	p>	></o:p>	NULL<o:p></o:p> >	p>	ID<o:p></o:p>
2<o:p>< </o:p> >	user_id<o:p></o:p>	BIGINT<o:p></o:p>	FK → users, NOT NULL<o:p></o:p> >	-<o:p></o:p>	피드백 제출 사용 자<o:p></o:p>
3<o:p>< </o:p> >	summary_id<o:p></o:p> p>	BIGINT<o:p></o:p>	FK → summaries, NOT NULL<o:p></o:p> >	-<o:p></o:p>	대상 요약 카드 <o:p></o:p>
4<o:p>< </o:p> >	type<o:p></o:p>	VARCHAR(10)<o:p></o:p>	NOT NULL<o:p></o:p> >	-<o:p></o:p>	LIKE 또는 DISLIKE<o:p></o:p>
5<o:p>< </o:p> >	created_at<o:p></o:p> >	TIMESTAMP<o:p></o:p>	NOT NULL<o:p></o:p> >	NOW()<o:p></o:p>	피드백 제출 일시 <o:p></o:p>

<o:p> </o:p>

(7)DD-07, user_preferences – 선호도 모델<o:p></o:p>

--	--	--	--	--	--

번호 <o:p></o:p>	필드 명<o:p></o:p>	자료타입 <o:p></o:p>	값의 범주 <o:p></o:p>	초기 값 <o:p></o:p>	비고 <o:p></o:p>
1<o:p></o:p>	preference_id<o:p></o:p>	BIGSERIAL<o:p></o:p>	PK, NOT NULL<o:p></o:p>	auto<o:p></o:p>	순서도 모델 고유 ID<o:p></o:p>
2<o:p></o:p>	user_id<o:p></o:p>	BIGINT<o:p></o:p>	FK → users, UNIQUE<o:p></o:p>	-<o:p></o:p>	대상 사용자 (1:1)<o:p></o:p>
3<o:p></o:p>	topic_weights<o:p></o:p>	JSONB<o:p></o:p>	NULL<o:p></o:p>	{}<o:p></o:p>	주제별 선호 가중 치 맵 (LangChain 학습 결과)<o:p></o:p>
4<o:p></o:p>	updated_at<o:p></o:p>	TIMESTAMP<o:p></o:p>	NOT NULL<o:p></o:p>	NOW()<o:p></o:p>	마지막 선호도 갱 신 일시 <o:p></o:p>

(8)DD-08, notifications – 알림 설정<o:p>

번호	필드 명	자료타입	값의 범주	초기 값	비 고
1	notification_id	BIGSERIAL	PK, NOT NULL	auto	알림 설정 고유 ID
2	user_id	BIGINT	FK → users, UNIQUE	—	대상 사용자 (1:1)
3	scheduled_time	TIME	NOT NULL	08:00	사용자 지정 푸시 발송 시각
4	is_enabled	BOOLEAN	NOT NULL	TRUE	알림 활성화 여부

6. 구현 기술 설계

구분	클라이언트	서버
구현 언어	- JavaScript(React Native / Expo)	- Java 17, Python(LangChain)
프레임워크	- React Native, Expo	- Spring Boot 3.2
운영체제	- iOS, Android	- Linux(AWS EC2 Ubuntu)
데이터베이스		- PostgreSQL (AWS RDS)
특수 소프트웨어	- Expo EAS Build, Firebase Cloud Messaging	- Docker, Github Actions CI/CD
하드웨어	- iPhone 16 Pro	- AWS EC2, AWS RDS PostgreSQL AWS S3
네트워크	- HTTPS / Wi-Fi / LTE	- HTTPS/TLS, JDBC(SSL), AWS VPC

7. 요구사항 추적표

요구사항식별자	M-001	M-002	M-003	M-004	M-005	M-006	M-007
FR-001					●		
FR-002					●		
FR-003							●
FR-004					●		
FR-005					●		
FR-006		●					
FR-007							●
FR-008		●					
FR-009		●					
FR-010		●					●
FR-011			●				
FR-012			●				
FR-013				●		●	
FR-014				●		●	
FR-015	●						
FR-016	●						

8. 부록

해당사항 없음